MoPilot

KI-gestützter Mobilitätsassistent für E-Carsharing im ländlichen Raum

Prototyp v0.4 Datum: 3. März 2026

Inhaltsverzeichnis

1. Zusammenfassung
2. IT-Umfeld
   - 2.1 Genutzte Werkzeuge und Technologien
   - 2.2 Server-Infrastruktur
   - 2.3 Admin und Deployment
   - 2.4 IT-Übersichtsgrafik
   - 2.5 IT-Detailgrafik
3. Deployment-Prozess (CI/CD)
   - 3.1 Übersicht
   - 3.2 Komponenten der Pipeline
   - 3.3 Ablauf eines Deployments
   - 3.4 GitHub Actions Workflow
   - 3.5 Deployment-Diagramm
4. Nächste Schritte
5. ZEO Kundenassistent – Entwicklung und Deployment (zeo-kunden.mopilot.website)
   - 5.1 Projektbeschreibung und Ziel
   - 5.2 Technische Architektur
   - 5.3 Entwicklungsprozess mit Claude Code
   - 5.4 Deployment-Ablauf: Herausforderungen und Lösungen
   - 5.5 Live-System und Ergebnis
6. Nächstes Sub-Projekt: hotline.mopilot.website
   - 6.1 Konzept und Zielgruppe
   - 6.2 Technische Anforderungen
   - 6.3 Entwicklungsschritte
   - 6.4 Deployment-Checkliste
   - 6.5 Zeitplan

1. Zusammenfassung

MoPilot ist ein KI-gestützter Mobilitätsassistent für E-Carsharing im ländlichen Raum. Der Prototyp v0.4 demonstriert einen intelligenten Assistenten, der Tonalität, Wissen und Funktionen dynamisch an die jeweilige Nutzerrolle anpasst.

Projektkontext

E-Carsharing im ländlichen Raum steht vor besonderen Herausforderungen: geringe Bevölkerungsdichte, weite Entfernungen und heterogene Nutzergruppen. MoPilot adressiert diese Herausforderungen durch einen KI-Assistenten auf Basis von Anthropics Claude (Sonnet), der als zentrale Schnittstelle zwischen Nutzern, Betreibern und strategischen Partnern fungiert.

Zugangschutz

Alle MoPilot-Websites sind durch einen vorgelagerten Passwortschutz (Basic Auth via Next.js Middleware) gesichert. Die Demo-Zugangsdaten gelten für alle Sub-Projekte:

Feld	Wert
Benutzer	`MoPilot`
Passwort	`mopilot2027`

Geschützte URL	Zugangsstufe
https://mopilot.website	Basic Auth → rollenbasierter App-Login
https://zeo-kunden.mopilot.website	Basic Auth → direkte App-Nutzung
https://hotline.mopilot.website	Basic Auth (geplant, intern)

Nach erfolgreicher Eingabe des Passwortschutzes ist die jeweilige Anwendung erreichbar. Bei mopilot.website folgt zusätzlich der rollenbasierte Login (siehe Rollensystem unten).

Rollensystem

Der Prototyp implementiert 10 verschiedene Nutzerrollen in 3 Kategorien. Jede Rolle erhält einen individuell angepassten KI-Kontext:

Kategorie	Rolle	E-Mail	Beschreibung
Kundennah	Endkunde	endkunde@mopilot.website	Buchung, FAQs, Standorte
	Stationspate	stationspate@mopilot.website	Standortbetreuung, Meldungen
	Hotline	hotline@mopilot.website	Gesprächsleitfaden, Kundenhilfe
Betrieb	Betreiber	betreiber@mopilot.website	Dashboard, Kennzahlen, Strategie
	Flottenmanagement	flotte@mopilot.website	Fahrzeuge, Wartung, Zuweisung
	Fahrzeugbetreuer	fahrzeug@mopilot.website	Zustandsprüfung, Laden
	Plattform-Support	support@mopilot.website	Technik, Tickets, Systeme
Strategie	Projektträger	traeger@mopilot.website	KPIs, Förderung, Strategie
	Fahrzeugsteller	steller@mopilot.website	Fahrzeugintegration, Verträge
	Validierungsstelle	validierung@mopilot.website	Führerscheinprüfung, Dokumente

Funktionsumfang v0.4

mopilot.website (Hauptsystem)

- Neues Header-Design mit integrierten Logos (MoPilot, ZEO Carsharing, Car&RideSharing Community eG)

- Vorgelagerter Passwortschutz (Basic Auth) für mopilot.website
- Rollenbasierter Login mit 12 Demo-Accounts (10 ZEO + 2 CC)
- KI-Chat mit Claude Sonnet – rollenspezifische Tonalität und Wissen
- REST-API mit JWT-Authentifizierung (8h Token-Gültigkeit)
- Wissensbasis mit FAQs pro Betreiber und Rolle
- Standort-, Fahrzeug- und Tarifverwaltung
- Swagger UI für API-Dokumentation
- Vollständige CI/CD-Pipeline mit GitHub Actions und automatischem Deployment

zeo-kunden.mopilot.website (ZEO Kundenassistent)

- Neues Header-Design mit integrierten Logos (ZEO Carsharing, MoPilot)
- Vorgelagerter Passwortschutz (Basic Auth) – neu in v0.4
- KI-Chat mit SSE-Streaming (wortweise Ausgabe)
- Kostenrechner, ÖPNV-Karte, Reichweiten-Guide, Fahrzeugkatalog, Onboarding

---

2. IT-Umfeld

2.1 Genutzte Werkzeuge und Technologien

Bereich	Technologie	Details
Frontend	Next.js 14	React-basiert, App Router, SSR, Node 20 Alpine
	Tailwind CSS	Utility-First CSS Framework
	Leaflet.js	Interaktive Karte (OpenStreetMap) für ZEO-Stationen
Backend	FastAPI	Python 3.11, async, Uvicorn mit 2 Workers
	Anthropic SDK	Python-Bibliothek für Claude API-Kommunikation
	SQLAlchemy 2.0	Async ORM mit asyncpg Driver
	Pydantic v2	Datenvalidierung und Settings-Management
	Paramiko	Python SFTP-Bibliothek für direktes Server-Deployment
KI-Modell	Claude Sonnet	`claude-sonnet-4-6` via Anthropic API
Datenbank	PostgreSQL 16	Alpine-Image, persistentes Volume
Cache	Redis 7	Alpine-Image, 256MB, LRU-Eviction
Auth	JWT (python-jose)	8h Token, bcrypt Passwort-Hashing
Zugangschutz	Basic Auth	Next.js Middleware, gilt für alle Sub-Projekte
Containerisierung	Docker	Multi-Stage Builds, Docker Compose
Reverse Proxy	Traefik	Via Coolify, SSL-Terminierung, Host-basiertes Routing
SSL/TLS	Let's Encrypt	Automatische Zertifikate via Traefik (certresolver)
Deployment	Coolify	Self-hosted PaaS, verwaltet Traefik + Container
CI/CD	GitHub Actions	Automatisches Deployment bei Push auf master
Versionskontrolle	Git + GitHub	Private Repository, Branch-basierter Workflow
DNS	IONOS	Domain-Verwaltung für mopilot.website (A-Records)
API-Dokumentation	Swagger UI	Auto-generiert durch FastAPI unter /docs
KI-Entwicklung	Claude Code	KI-gestütztes Pair Programming (lokal, CLI)

2.2 Server-Infrastruktur

Eigenschaft	Wert
Anbieter	Hetzner Cloud
Server-Typ	CX33 (Shared vCPU)

IPv4	142.132.232.211
IPv6	2a01:4f8:c17:b65c::1
Standort	Falkenstein (FSN1), Deutschland
Betriebssystem	Ubuntu (Docker CE)
SSH-Zugang	root@142.132.232.211 (Passwort-Auth)
Frontend-Domain (Hauptsystem)	https://mopilot.website
Backend-Domain (Hauptsystem)	https://api.mopilot.website
API-Dokumentation	https://api.mopilot.website/docs
Frontend-Domain (ZEO Kunden)	https://zeo-kunden.mopilot.website
Backend-Domain (ZEO Kunden)	https://api.zeo-kunden.mopilot.website
Frontend-Domain (Hotline, geplant)	https://hotline.mopilot.website
Coolify-Dashboard	http://localhost:8000 (via SSH-Tunnel)
GitHub-Repository	github.com/JamesMz05/mopilot (privat)

2.3 Admin und Deployment

Das Deployment erfolgt über Coolify, ein self-hosted Platform-as-a-Service (PaaS), das auf dem Hetzner-Server läuft. Coolify verwaltet:

- Docker Compose Service-Stacks (Build, Deploy, Restart)
- Reverse Proxy (automatisches SSL/TLS via Let's Encrypt)
- Environment Variables (Secrets wie API-Keys, DB-Passwörter)
- Container-Logs und Terminal-Zugriff
- Health Checks und Auto-Restart

Docker Compose Services

Container	Image/Build	Port	Funktion
frontend	Next.js (Node 20 Alpine)	3000	Web-Oberfläche, SSR, Basic Auth Middleware
backend	FastAPI (Python 3.11 Slim)	8000	REST-API, KI-Chat
postgres	postgres:16-alpine	5432	Relationale Datenbank
redis	redis:7-alpine	6379	Cache, Session-Store

2.4 IT-Übersichtsgrafik

```
```

```
+==============================================================+
|                NUTZER (Browser)                              |
|        mopilot.website / api.mopilot.website                 |
+==================+===============+=========================+
              |            |
          HTTPS|        HTTPS|
              v            v
+==============================================================+
|           HETZNER CX33  (142.132.232.211)                    |
| +-----------------------------------------------------+  |
| |            COOLIFY  (Reverse Proxy)                 |  |
| |        SSL/TLS Termination + Routing                |  |
| +---------------+----------------+------------------+  |
|              |              |                  |
|   +--------------+--+   +---------+---------+            |
|   |  FRONTEND      |   |    BACKEND        |            |
|   |  Next.js :3000 |   |   FastAPI :8000   |            |
|   |  Basic Auth    |   |   (Python 3.11)   |            |
|   |  (Node 20)     |   |                   |            |
|   +----------------+   +----+---------+----+            |
|                        |        |              |
|               +----------+--+  +---+--------+        |
|               | POSTGRESQL  |  |   REDIS    |        |
|               | :5432       |  |   :6379    |        |
|               +-------------+  +------------+        |
|                                          |
|   +-----------------------------------------------+    |
|   |        ANTHROPIC CLOUD API (extern)           |    |
|   |        Claude Sonnet - KI-Modell              |    |
|   +-----------------------------------------------+    |
+==============================================================+
```

2.5 IT-Detailgrafik

```
+--[ Browser ]-----------------------------------------+
|  Aufruf mopilot.website  -->  Basic Auth Prompt         |
|  User: MoPilot / PW: ****  -->  Zugang gewährt          |
|  Klick auf Rolle         -->  POST /api/auth/login      |
|  JWT gespeichert         -->  POST /api/chat/send       |
+-----+-----------------------------------------------+
     |  HTTPS (JWT Bearer Token)
```

```
    v
+--[ Coolify Reverse Proxy ]-----------------------------+
|  mopilot.website      -->  frontend:3000                |
|  api.mopilot.website  -->  backend:8000                 |
+-----+-----------------------------------------------------+
    |
    v
+--[ Next.js Frontend :3000 ]----------------------------+
|  Middleware: Basic Auth                                 |
|    User: MoPilot  /  PW: mopilot2027                    |
|    -> Zugang zu Rollen-Login und Chat-UI                |
+-----+-----------------------------------------------------+
    |
    v
+--[ FastAPI Backend :8000 ]-----------------------------+
|                                                         |
|  /api/auth/login     -->  [ auth.py ]                   |
|    Email + Passwort  -->  bcrypt verify                 |
|    Erfolg            -->  JWT Token (8h)                |
|                                                         |
|  /api/chat/send      -->  [ chat.py ]                   |
|    User-Nachricht    -->  Rollen-Kontext laden          |
|    + Chat-History    -->  Anthropic API aufrufen        |
|    Claude-Antwort    <--  Antwort speichern + senden    |
|                                                         |
|  /api/knowledge/     -->  [ knowledge.py ]              |
|  /api/stations/      -->  [ stations.py ]               |
|  /api/vehicles/      -->  [ vehicles.py ]               |
|  /api/tariffs/       -->  [ tariffs.py ]                |
|                                                         |
+------+----------------------+---------------------------+
     |                    |
     v                    v
+--[ PostgreSQL ]---+  +--[ Anthropic Cloud ]---+
|  Users           |  |  Claude Sonnet         |
|  ChatMessages    |  |  Rollenspezifischer    |
|  Stations        |  |  System-Prompt         |
|  Vehicles        |  |  + Chat-History        |
|  Tariffs         |  |  + Wissensbasis        |
|  FAQs            |  +------------------------+
+-------------------+
```

---

3. Deployment-Prozess (CI/CD)

3.1 Übersicht

MoPilot verfügt über eine vollautomatische CI/CD-Pipeline (Continuous Integration / Continuous Deployment), die den gesamten Weg vom lokalen Code bis zur produktiven Website automatisiert. Jede Code-Änderung, die auf den master-Branch gepusht wird, löst automatisch ein vollständiges Deployment aus – ohne manuellen Eingriff.

Der gesamte Prozess dauert in der Regel unter 2 Minuten und umfasst drei Phasen: Code-Synchronisation auf den Server, Docker-Container-Rebuild und Aktivierung der neuen Version.

3.2 Komponenten der Pipeline

| Komponente | Technologie | Aufgabe |

|---|---|---|

| Versionskontrolle | Git + GitHub | Quellcode-Verwaltung, privates Repository unter github.com/JamesMz05/mopilot |

| CI/CD-Runner | GitHub Actions | Automatisierte Pipeline, wird bei jedem Push auf master ausgelöst |

| Server-Zugriff | SSH (appleboy/ssh-action) | Sichere Verbindung zum Hetzner-Server für Code-Synchronisation |

| Container-Management | Coolify + Docker | Rebuild der Container-Images, Neustart der Services, Reverse Proxy |

Sicherheit: GitHub Secrets

Secret	Zweck

SERVER_SSH_PASSWORD	SSH-Passwort für den Zugriff auf den Hetzner-Server (root@142.132.232.211)
GH_PAT	GitHub Personal Access Token – erlaubt dem Server, Code vom Repository zu laden
COOLIFY_TOKEN	API-Token für Coolify – löst den Container-Rebuild aus

3.3 Ablauf eines Deployments

1. **Lokale Entwicklung**: Der Entwickler bearbeitet Code lokal in `C:\Projekte\MoPilot` und testet die Änderungen.

2. **Git Commit & Push**: Die Änderungen werden committet und auf den master-Branch gepusht:

```
git add . && git commit -m "..." && git push origin master
```

3. **GitHub Actions startet**: GitHub erkennt den Push auf master und startet automatisch den Workflow "Deploy to Coolify". Der Workflow läuft auf einem GitHub-eigenen Ubuntu-Runner.

4. **SSH-Verbindung zum Server**: Der Workflow verbindet sich per SSH mit dem Hetzner-Server (142.132.232.211) unter Nutzung des verschlüsselten SSH-Passworts.

5. **Code-Synchronisation**: Auf dem Server wird der neueste Code vom GitHub-Repository geladen: `git fetch` und `git reset --hard`. Damit ist der Server-Code identisch mit dem Repository.

6. **Coolify Rebuild**: Der Workflow löst über die Coolify-API einen vollständigen Rebuild aus. Coolify baut alle Docker-Container neu (Frontend und Backend).

7. **Container-Neustart**: Coolify stoppt die alten Container und startet die neuen Images. Der Reverse Proxy leitet den Traffic nahtlos auf die neue Version um.

8. **Live**: Die Änderungen sind unter mopilot.website und api.mopilot.website verfügbar. Die gesamte Pipeline dauert typischerweise 60–90 Sekunden.

3.4 GitHub Actions Workflow

```yaml
name: Deploy to Coolify

on:
  push:
    branches: [master]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Pull latest code on server
        uses: appleboy/ssh-action@v1
        with:
          host: 142.132.232.211
          username: root
          password: ${{ secrets.SERVER_SSH_PASSWORD }}
          script: |
            cd /opt/mopilot
            git fetch https://${{ secrets.GH_PAT }}@github.com/...
            git reset --hard FETCH_HEAD

      - name: Trigger Coolify Rebuild
        run: |
          curl -s -X GET \
            -H "Authorization: Bearer ${{ secrets.COOLIFY_TOKEN }}" \
            "https://coolify.pydna.de/api/v1/deploy?uuid=...&force=true"
```

> Hinweis: Sensible Werte (Passwörter, Tokens, URLs) werden durch GitHub Secrets ersetzt und sind im Code nur als `${{ secrets.NAME }}` Platzhalter sichtbar.

3.5 Deployment-Diagramm

```
+--[ Entwickler-PC ]-----+
|  C:\Projekte\MoPilot   |
|  Code bearbeiten       |
|  git commit + push     |
+----------+-------------+
           |
           | git push (HTTPS)
```

```
        v
+--[ GitHub ]-----------------------------------+
|  Repository: JamesMz05/mopilot (privat)       |
|  Branch: master                               |
|                                               |
|  +--[ GitHub Actions ]---------------------+  |
|  |  Trigger: push auf master               |  |
|  |  Runner: ubuntu-latest                  |  |
|  |                                         |  |
|  |  Step 1: SSH auf Server                 |  |
|  |    -> git fetch + reset                 |  |
|  |                                         |  |
|  |  Step 2: Coolify API aufrufen           |  |
|  |    -> Container Rebuild starten         |  |
|  +-----------------------------------------+  |
+-------+--------------------+------------------+
      |                    |
  SSH |            HTTP    |
      v                    v
+--[ Hetzner Server ]-------------------------------+
|  142.132.232.211                                 |
|                                                  |
|  /opt/mopilot/        <-- Code aktualisiert      |
|    frontend/                                     |
|    backend/                                      |
|    docker-compose.yml                            |
|                                                  |
|  +--[ Coolify ]------------------------------+   |
|  |  Docker Images neu bauen                  |   |
|  |  Container stoppen + starten              |   |
|  |  Reverse Proxy aktualisieren              |   |
|  +-------------------------------------------+   |
|                                                  |
|  +--[ Live ]---------------------------------+   |
|  |  mopilot.website       (Frontend)         |   |
|  |  api.mopilot.website   (Backend/API)      |   |
|  +-------------------------------------------+   |
+--------------------------------------------------+
```

---

4. Nächste Schritte

Git-Repository und CI/CD-Pipeline

Status: Abgeschlossen am 28.02.2026

Das GitHub-Repository (github.com/JamesMz05/mopilot) wurde eingerichtet und eine vollautomatische CI/CD-Pipeline mit GitHub Actions konfiguriert. Jeder Push auf den master-Branch löst automatisch ein Deployment auf dem Produktionsserver aus. Details zur Pipeline finden sich in Kapitel 3.

Schritt 2: SSE-Streaming im Chat aktivieren

Priorität: Hoch | Aufwand: 1 Tag

Das Backend hat bereits einen `/api/chat/stream` Endpoint (Server-Sent Events), aber das Frontend nutzt aktuell nur den synchronen `/api/chat/send` Endpoint. Durch SSE-Streaming erscheinen KI-Antworten wortweise – deutlich bessere UX.

- Frontend Dashboard um EventSource/fetch-Stream-Handling erweitern
- Typing-Indicator und wortweises Rendering implementieren
- Fehlerbehandlung für abgebrochene Streams einbauen
- Fallback auf synchronen Endpoint bei Stream-Fehler

Schritt 3: Rollenspezifische System-Prompts verfeinern

Priorität: Mittel | Aufwand: 2–3 Tage

Die Kernfunktion von MoPilot ist die rollenbasierte KI-Anpassung. Die System-Prompts müssen mit echtem Domainwissen aus dem ZEO-Betrieb angereichert und mit den Betreibern validiert werden.

- Workshops mit ZEO-Betreibern für rollenspezifische Anforderungen
- System-Prompts mit echten Tarifen, Standorten und Prozessen füllen
- Prompt-Evaluation mit realistischen Nutzeranfragen durchführen
- Feedback-Mechanismus im Chat für Prompt-Verbesserung einbauen

Schritt 4: Monitoring, Logging und Stabilität

Priorität: Mittel | Aufwand: 2 Tage

- Health-Check Endpoints für alle Services mit Auto-Restart in Coolify
- Uptime-Monitoring einrichten (z.B. Uptime Kuma, ebenfalls self-hosted)
- Strukturiertes Logging mit Log-Rotation implementieren
- Alerting bei Ausfällen (E-Mail oder Webhook)
- Docker Restart-Policy auf "unless-stopped" setzen

Schritt 5: Nutzertests und UI/UX-Verbesserungen

Priorität: Mittel | Aufwand: 3–5 Tage

- Chat-Verlauf persistent speichern und beim Reload wiederherstellen
- Mobile-Responsive Design optimieren (aktuell nur Desktop)
- Markdown-Rendering für KI-Antworten (Listen, Code, Links)
- Dark Mode und barrierefreie Gestaltung (WCAG 2.1)
- Erste Nutzertests mit 2–3 Personen pro Rollenkategorie durchführen

Zeitplan-Übersicht

Schritt	Aufwand	Priorität	Status
1. Git + CI/CD	1–2 Tage	Hoch	Erledigt
2. SSE-Streaming	1 Tag	Hoch	Offen
3. System-Prompts	2–3 Tage	Mittel	Offen
4. Monitoring	2 Tage	Mittel	Offen
5. UX + Nutzertests	3–5 Tage	Mittel	Offen

Verbleibender Aufwand: 8–11 Arbeitstage

---

5. ZEO Kundenassistent – Entwicklung und Deployment

Im Rahmen des MoPilot-Projekts wurde ein eigenständiger, KI-gestützter Kundenassistent für ZEO E-Carsharing entwickelt und unter https://zeo-kunden.mopilot.website in Betrieb genommen.

5.1 Projektbeschreibung und Ziel

Während MoPilot (mopilot.website) ein rollenbasiertes Demo-System für verschiedene Carsharing-Akteure ist, richtet sich der ZEO Kundenassistent ausschließlich an Endkunden von ZEO Carsharing. Das Ziel war ein informatives und KI-gestütztes Tool, das alle ZEO-relevanten Fragen beantwortet. Ab v0.4 ist auch diese App durch einen vorgelagerten Passwortschutz (Basic Auth) gesichert – die Zugangsdaten entsprechen dem Hauptsystem (User: `MoPilot` / PW: `mopilot2027`).

Die App besteht aus fünf interaktiven Modulen auf einer Single-Page:

- **KI-Chat (Hero-Element)**: SSE-Streaming-Assistent mit ZEO-Systemkontext und Beispielfragen

- **Modul 1 – Kostenrechner**: Drei Schieberegler errechnen den optimalen Tarif (eco vs. eco plus)

- **Modul 2 – ÖPNV-Karte**: Leaflet/OpenStreetMap mit 12 ZEO-Stationen und ÖPNV-Anbindung

- **Modul 3 – Reichweiten-Guide**: Temperaturabhängige Reichweitenberechnung für alle 9 Fahrzeugmodelle

- **Modul 4 – Fahrzeugkatalog**: Filterbare Kacheln mit Specs, Eignung und Reichweite

- **Modul 5 – Onboarding**: 6-Schritt-Guide und FAQ-Akkordeon mit 18 häufigen Fragen

5.2 Technische Architektur

```
```

```
+==============================================================+
|            NUTZER (Browser)                                  |
|   https://zeo-kunden.mopilot.website                         |
+==========================+=================================+
                    |  HTTPS
                    v
+==============================================================+
|        HETZNER CX33  (142.132.232.211)                       |
|                                                              |
| +-----------------------------------------------------+  |
| |  COOLIFY-PROXY (Traefik) – Port 80/443              |  |
| |  SSL/TLS via Let's Encrypt (automatisch)            |  |
| +-------------+--------------------------+-----------+  |
|             |                        |             |
| +-------------+----------+  +--------------+-----------+   |
| |  FRONTEND  :3000       |  |  BACKEND  :8000        |  |
| |  Next.js 14 (Node 20)  |  |  FastAPI (Python 3.11)   |  |
| |  TailwindCSS           |  |  Anthropic SDK          |  |
| |  Leaflet.js            |  |  SSE-Streaming          |  |
| |  Basic Auth Middleware |  |  ZEO System-Prompt       |  |
| |  Static JSON Data      |  |                         |  |
| +------------------------+  +-------------------------+   |
|                                                              |
| +-----------------------------------------------------+  |
| |  ANTHROPIC CLOUD API (extern)                       |  |
| |  Claude claude-sonnet-4-6 – KI-Modell               |  |
| +-----------------------------------------------------+  |
+==============================================================+
```

Komponente	Technologie	Begründung
Frontend	Next.js 14, App Router, TailwindCSS	Identisch mit mopilot.website – Wiederverwendung von Know-how
Zugangschutz	Basic Auth (Next.js Middleware)	Einheitlicher Schutz aller MoPilot-Sub-Projekte
Karte	Leaflet.js + OpenStreetMap	Keine API-Kosten, Open Source, datenschutzkonform
Backend	FastAPI (Python 3.11)	Leichtgewichtig, SSE-Streaming nativ unterstützt
KI-Modell	Claude claude-sonnet-4-6	Hohe Qualität, deutschsprachig, Anthropic API
Datenhaltung	Statische JSON-Dateien	Kein Datenbank-Overhead für Prototyp
Reverse Proxy	Traefik (via Coolify)	Automatisches Routing und SSL-Terminierung
SSL	Let's Encrypt (via Traefik)	Automatisch, kostenlos, kein Wartungsaufwand
Deployment	Docker Compose + Traefik-Labels	Integration in bestehende Coolify-Infrastruktur

5.3 Entwicklungsprozess mit Claude Code

Die gesamte Applikation wurde in einer einzigen Session mit Claude Code entwickelt. Der Prozess war vollständig iterativ und dokumentengetrieben.

Anforderungsanalyse

Aus den vier PDF-Dateien im Ordner `MoPilot_ZEO_Kunden\` wurden alle relevanten ZEO-Inhalte extrahiert und in JSON-Dateien strukturiert:

- `vehicles.json`: 9 Fahrzeugmodelle mit Specs (Sitze, Reichweite, Kofferraum, Ausstattung)
- `tariffs.json`: Beide Tarife (eco, eco plus) mit allen Preisdetails
- `stations.json`: 12 Musterstationen mit Koordinaten und ÖPNV-Anbindung
- `faq.json`: 18 häufige Fragen in 3 Kategorien

Code-Generierung

Claude Code generierte alle Projektdateien in einem Schritt: Backend (FastAPI), Frontend (Next.js mit 8 Komponenten), Docker-Konfiguration und Nginx-Referenzkonfiguration. Insgesamt wurden 35 Dateien mit 2.736 Codezeilen erstellt.

Git-Workflow

Der Code wurde auf einem neuen Feature-Branch (`feature/zeo-kundenassistent`) committet, ein Pull Request erstellt und nach Review in den master-Branch gemergt.

5.4 Deployment-Ablauf: Herausforderungen und Lösungen

Phase 1: Coolify GitHub-Integration (Problem)

```

Failed to read Git source: fatal: could not read Username for

  'https://github.com': No such device or address

```

Ursache: Das GitHub-Repository ist privat. Die GitHub App-Konfiguration erwies sich als zu komplex für den Prototypen-Kontext.

Phase 2: Direktes Deployment per SSH/SCP (Lösung)

```python
# Python-Script mit paramiko:
1. SSH-Verbindung zum Server herstellen
2. Alle 31 Projektdateien per SFTP auf /opt/zeo-kunden/ übertragen
3. .env-Datei mit ANTHROPIC_API_KEY auf dem Server erstellen
4. docker compose up -d --build ausführen
```

Phase 3: Traefik-Integration (Reverse Proxy)

```yaml
networks:
  coolify:
    external: true   # Verbindung zum Traefik-Netzwerk

labels:
  - "traefik.enable=true"
  - "traefik.http.routers.zeo-frontend.rule=Host(`zeo-kunden.mopilot.website`)"
  - "traefik.http.routers.zeo-frontend.tls.certresolver=letsencrypt"
  - "traefik.http.services.zeo-frontend.loadbalancer.server.port=3000"
```

Phase 4: DNS-Konfiguration

```
zeo-kunden.mopilot.website     A  142.132.232.211  TTL: 1 Min.
api.zeo-kunden.mopilot.website A  142.132.232.211  TTL: 1 Min.
```

Phase 5: API-Key-Rotation

Nach dem ersten erfolgreichen Deployment wurde der Anthropic API-Key rotiert und per SSH-Script in der `.env`-Datei auf dem Server aktualisiert:

```bash
docker compose restart backend
```

5.5 Live-System und Ergebnis

```bash
$ curl https://api.zeo-kunden.mopilot.website/api/health
{"status": "ok", "service": "zeo-kundenassistent"}
```

```
NAME                          IMAGE                  STATUS
mopilot_zeo_kunden-frontend-1   zeo_kunden-frontend      Up
mopilot_zeo_kunden-backend-1    zeo_kunden-backend       Up
```

Ort	Pfad / URL
Lokal (Entwicklung)	C:\Projekte\MoPilot\MoPilot_ZEO_Kunden\
GitHub (Versionierung)	github.com/JamesMz05/mopilot (Branch: master)
Server (Deployment)	/opt/zeo-kunden/MoPilot_ZEO_Kunden/
Frontend (live)	https://zeo-kunden.mopilot.website
Backend/API (live)	https://api.zeo-kunden.mopilot.website/api/health

---

6. Nächstes Sub-Projekt: hotline.mopilot.website

6.1 Konzept und Zielgruppe

hotline.mopilot.website richtet sich an die Mitarbeiterinnen und Mitarbeiter der ZEO Carsharing Hotline (betrieben durch Vianova eG). Es ist ein internes Arbeitsmittel:

- **Zielgruppe**: Hotline-Mitarbeiter (kein öffentlicher Zugang, kein Login für Endkunden)
- **Kernfunktion**: KI-gestützter Gesprächs-Assistent, der bei eingehenden Anrufen in Echtzeit unterstützt
- **Wissensquelle**: Vollständige ZEO-Wissensbasis (Tarife, Fahrzeuge, Stationen, Prozesse, Sonderfälle)
- **Zusatzfunktionen**: Gesprächsleitfaden, Eskalationspfade, häufige Problemszenarien

6.2 Technische Anforderungen

Anforderung	Beschreibung	Priorität
Zugangsschutz	Basic Auth oder einfacher Login-Schutz (intern)	Hoch
Schnelle Antworten	Antwortzeit < 3 Sekunden, SSE-Streaming	Hoch
Gesprächs-Kontext	Kontextgedächtnis: laufendes Gespräch als Session	Hoch
Geräteoptimierung	Optimiert für Desktop-Browser (Headset-Arbeitsplatz)	Mittel
Offline-Fallback	Wichtigste Infos auch ohne Internet abrufbar	Niedrig
Datenaktualität	Einfache Pflegbarkeit der Wissensbasis (JSON im Git)	Mittel

6.3 Entwicklungsschritte

Schritt 1: Inhaltsvorbereitung und System-Prompt (0,5 Tage)

- Vollständige ZEO-Wissensbasis zusammenstellen
- Hotline-spezifischen System-Prompt entwickeln
- Gesprächs-Szenarien definieren
- JSON-Datensätze aus ZEO-Kunden-App übernehmen und erweitern

Schritt 2: Projektstruktur anlegen (0,5 Tage)

```
C:\Projekte\MoPilot\MoPilot_Hotline\
  frontend/    # Next.js 14
  backend/     # FastAPI (aus ZEO-App kopieren, System-Prompt anpassen)
  docker-compose.yml
  .env.example
```

Schritt 3: Frontend-Entwicklung (1–2 Tage)

- Kompaktes, zweispaltiges Layout: Links Chat, rechts Schnellreferenz-Panel
- Schnellzugriff-Buttons: häufigste Anrufgründe als vorgefertigte Fragen
- Session-Protokoll: laufendes Gespräch wird angezeigt und kann exportiert werden
- Gesprächs-Leitfaden: interaktiver Schritt-für-Schritt-Guide
- Dunkel-Modus für Bildschirmarbeit am Headset-Arbeitsplatz
- Basic Auth Schutz via Next.js Middleware

Schritt 4: Backend-Anpassung (0,5 Tage)

- System-Prompt auf Hotline-Rolle anpassen
- CORS-Origins auf hotline.mopilot.website beschränken
- Optional: Session-ID im Request für Gesprächsgedächtnis

Schritt 5: Deployment (0,5 Tage)

```bash
# 1. DNS bei IONOS
hotline.mopilot.website     A  142.132.232.211
api.hotline.mopilot.website A  142.132.232.211

# 2. Dateien per SFTP auf Server
/opt/hotline/MoPilot_Hotline/
```

```
# 3. Traefik-Labels in docker-compose.yml
traefik.http.routers.hotline-frontend.rule=Host(`hotline.mopilot.website`)

# 4. Start
docker compose up -d --build

# 5. SSL automatisch via Traefik/Let's Encrypt
```

6.4 Deployment-Checkliste

- [ ] Inhalt: ZEO-Wissensbasis vollständig und aktuell
- [ ] Inhalt: System-Prompt mit Hotline-Mitarbeitern getestet
- [ ] Technik: DNS-Einträge bei IONOS gesetzt und propagiert
- [ ] Technik: Traefik-Labels in docker-compose.yml korrekt
- [ ] Technik: ANTHROPIC_API_KEY in .env gesetzt (separater Key empfohlen)
- [ ] Technik: CORS_ORIGINS auf hotline.mopilot.website beschränkt
- [ ] Sicherheit: Basic Auth Passwort gesetzt und nur intern kommuniziert
- [ ] Test: Chat-Funktion mit Beispiel-Anrufszenarien getestet
- [ ] Test: SSL-Zertifikat aktiv (https:// erreichbar)
- [ ] Test: API Health-Check positiv: `/api/health -> {"status": "ok"}`
- [ ] Abnahme: Demo mit Hotline-Team durchgeführt und Feedback eingearbeitet

6.5 Zeitplan

Phase	Aufwand	Voraussetzung
Inhaltsvorbereitung & System-Prompt	0,5 Tage	Abstimmung mit ZEO Hotline-Team
Projektstruktur & Code-Basis	0,5 Tage	ZEO-Kunden-App als Vorlage
Frontend-Entwicklung	1–2 Tage	Hotline-UX-Anforderungen bekannt
Backend-Anpassung	0,5 Tage	System-Prompt fertiggestellt
Deployment & Test	0,5 Tage	DNS & Server-Zugang verfügbar
GESAMT	**3–4 Tage**	

Durch die Wiederverwendung der ZEO-Kunden-App-Architektur reduziert sich der Aufwand erheblich. Der kritische Pfad liegt in der Inhaltsvorbereitung und der Abstimmung des System-Prompts mit dem Hotline-Team.

Erstellt mit Claude Code | Aktualisiert auf v0.4 am 3. März 2026